



NumPy Arrays Documentation

Arrays in NumPy are faster than Python lists because they store elements of the same data type in contiguous memory locations, enabling efficient operations and vectorization.

Core Concepts

- **Array Definition:** A fundamental linear data structure holding elements of the same type in contiguous memory.
- **Type Promotion Rules:**
 - Integers promote to float if any float is present.
 - Floats promote to string if any string is present.

1D Arrays (Vectors)

```
import numpy as np

# Create 1D array
a = np.array([1, 2, 3, 4, 5])
print(a)          # [1 2 3 4 5]
print(a.ndim)     # 1
print(type(a))    # <class 'numpy.ndarray'>
```

Type promotion example:

```
b = np.array([1, 2, 3, 4, 3.5]) # int → float
print(b) # [1. 2. 3. 4. 3.5]

c = np.array([3.5, "edwin"])     # float → str
print(c) # ['3.5' 'edwin']
print(c.size) # 2
```

2D Arrays (Matrices)

Structure: Rows × Columns

```
# 3x2 array
ab = np.array([[1, 2], [2, 3], [3, 4]])
print(ab)
print(ab.shape) # (3, 2)
print(ab.ndim)  # 2
print(ab[1][1]) # 3 (row 1, column 1)
```

```
# 3x3 array
abc = np.array([[1, 2, 2], [2, 1, 1], [3, 4, 5]])
print(abc.shape)    # (3, 3)
print(type(abc))    # <class 'numpy.ndarray'>
```

Example: 2x4 array

```
q2 = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
print(q2.shape)    # (2, 4)
```

3D Arrays (Tensors)

Structure: Depth (layers) x Rows x Columns

```
# 3x2x2 array
bn = np.array([[[1, 2], [3, 4]],
               [[5, 6], [7, 8]],
               [[9, 10], [11, 12]]])
print(bn.shape)    # (3, 2, 2)
print(bn.ndim)     # 3
```

Practice Examples:

Shape	Code Example
(3,3,3)	q1 = np.array([[[1,2,3],[4,5,6],[7,8,9]], [[5,4,7],[7,8,9],[5,6,8]], [[5,7,8],[8,8,8],[1,2,3]]])
(2,3,3)	q3 = np.array([[[2,3,4],[4,5,6],[7,8,9]], [[4,5,6],[4,7,8],[8,9,7]]])
(3,2,2)	q4 = np.array([[[2,1],[2,2]], [[5,6],[7,8]], [[4,5],[5,8]]])

Key Attributes Summary

Attribute	Description	Example
ndim	Number of dimensions	abc.ndim → 2
shape	Tuple of dimensions	ab.shape → (3, 2)
size	Total elements	c.size → 2
type()	Array type	<class 'numpy.ndarray'>

Quick Access Patterns

```
# Indexing (zero-based)
array[row][column] # 2D
# or
array[row, column] # 2D (preferred)
```

```
# Shape interpretation  
(3, 3, 3) # 3 layers, 3 rows, 3 columns each  
(2, 4)    # 2 rows, 4 columns
```

Save as `numpy_arrays.md` for reference!